

Trilha de Node.js

Módulo 02: Fundamentos

Instruções para a melhor prática de Estudo

1. **Leia atentamente todo o conteúdo:** Antes de iniciar qualquer atividade, faça uma leitura detalhada do material fornecido na trilha, compreendendo os conceitos e os exemplos apresentados.
2. **Não se limite ao material da trilha:** Utilize o material da trilha como base, mas busque outros materiais de apoio, como livros, artigos acadêmicos, vídeos, e blogs especializados. Isso enriquecerá o entendimento sobre o tema.
3. **Explore a literatura:** Consulte livros e publicações reconhecidas na área, buscando expandir seu conhecimento além do que foi apresentado. A literatura acadêmica oferece uma base sólida para a compreensão de temas complexos.
4. **Realize todas as atividades propostas:** Conclua cada uma das atividades práticas e teóricas, garantindo que você esteja aplicando o conhecimento adquirido de maneira ativa.
5. **Evite o uso de Inteligência Artificial para resolução de atividades:** Utilize suas próprias habilidades e conhecimentos para resolver os exercícios. O aprendizado vem do esforço e da prática.
6. **Participe de debates:** Discuta os conteúdos estudados com professores, colegas e profissionais da área. O debate enriquece o entendimento e permite a troca de diferentes pontos de vista.
7. **Pratique regularmente:** Não deixe as atividades para a última hora. Pratique diariamente e revise o conteúdo com frequência para consolidar o aprendizado.
8. **Peça feedback:** Solicite o retorno dos professores sobre suas atividades e participe de discussões sobre os erros e acertos, utilizando o feedback para aprimorar suas habilidades.

Essas instruções são fundamentais para garantir um aprendizado profundo e eficaz ao longo das trilhas.

Módulo 2: Fundamentos do Node.js

Objetivo do Módulo

Explorar os conceitos fundamentais do Node.js que capacitam os desenvolvedores a criar aplicações **eficientes, escaláveis e bem estruturadas**, compreendendo o funcionamento interno da plataforma, seus módulos nativos e os principais padrões de programação utilizados no mercado.

Ao final deste módulo, o aluno será capaz de:

- Compreender profundamente o funcionamento do Event Loop
- Trabalhar corretamente com programação assíncrona
- Organizar projetos utilizando módulos
- Manipular arquivos, streams e buffers
- Tratar erros e depurar aplicações
- Criar servidores HTTP básicos com Node.js puro

2.1: Módulos no Node.js: CommonJS vs ES Modules

O que são Módulos?

Os módulos são blocos reutilizáveis de código que ajudam na organização e manutenção de projetos.

CommonJS

Padrão histórico do Node.js.

- Importação com `require`
Exportação com `module.exports`
- Importação ocorre de forma síncrona no momento da execução

Exemplo:

```
// arquivo.js
module.exports = function() {
  console.log('Módulo CommonJS');
};

// app.js
const modulo = require('./arquivo');
modulo();
```

ES Modules (ESM)

Padrão moderno do JavaScript (ECMAScript).

- Importação com `import`
- Exportação com `export`
- Compatível com navegadores e projetos modernos

Exemplo:

```
// arquivo.mjs
export const saudacao = 'Olá, ES Modules!';

// app.mjs
import { saudacao } from './arquivo.mjs';
console.log(saudacao);
```

Diferenças Principais

Critério	CommonJS	ES Modules
Importação	<code>require</code>	<code>import</code>
Exportação	<code>module.exports</code>	<code>export</code>
Execução do import	Síncrona	Assíncrona
Compatibilidade	Node antigo	Node moderno e web
Uso recomendado	Projetos legados	Projetos novos

Quando usar:

1. Use CommonJS para aplicações legadas ou completamente server-side.
2. Prefira ESM para projetos modernos ou com compatibilidade web.

2.3: Manipulação de Arquivos e Diretórios com o módulo `fs`

Conceito

O módulo **fs (File System)** permite interagir com o sistema de arquivos, possibilitando a criação, leitura, atualização e exclusão de arquivos e diretórios.

Tipos de Operações

- **Síncronas:** bloqueiam o Event Loop (uso não recomendado)
- **Assíncronas:** não bloqueiam a execução (recomendado)

⚠ **Boas práticas:** Evite métodos síncronos em aplicações reais, pois eles prejudicam a performance.

Exemplos Práticos

Criar e Ler Arquivos:

```
const fs = require('fs');

// Criar arquivo
fs.writeFileSync('arquivo.txt', 'Conteúdo inicial');

// Ler arquivo
const conteudo = fs.readFileSync('arquivo.txt', 'utf8');
console.log(conteudo);
```

Assíncrono:

```
fs.writeFile('arquivo.txt', 'Conteúdo atualizado', (err) => {
  if (err) throw err;
  console.log('Arquivo criado com sucesso!');
});
```

Aplicabilidade

- Processamento de arquivos de log
 - Manipulação de uploads
 - Leitura de configurações
-

2.4: Trabalhando com Streams e Buffers

Conceito

Streams permitem processar dados em pequenos blocos (chunks), sendo ideais para arquivos grandes ou fluxos contínuos de dados, evitando alto consumo de memória.

Tipos de Streams

- **Readable:** leitura de dados
- **Writable:** escrita de dados
- **Duplex:** leitura e escrita
- **Transform:** transforma os dados durante o fluxo

Exemplo Prático

Leitura de Arquivo com Stream:

```
const fs = require('fs');

const stream = fs.createReadStream('grandeArquivo.txt', 'utf8');
stream.on('data', (chunk) => {
  console.log('Novo pedaço recebido:', chunk);
});
```

Buffers

Buffers armazenam temporariamente dados binários durante o processamento. São amplamente usados em streams.

Exemplo:

```
const buffer = Buffer.from('Node.js');
console.log(buffer.toString());
```

2.5: Tratamento de Erros e Estratégias de Depuração

Tipos de Erros

1. **Erros de Sintaxe:** Ocorrem durante a compilação.
2. **Erros de Runtime:** Ocorrem durante a execução do código.
3. **Erros Lógicos:** Resultados inesperados devido à lógica incorreta.

Métodos de Tratamento

1. Try/Catch:

```
try {
  const data = fs.readFileSync('inexistente.txt', 'utf8');
} catch (err) {
  console.error('Erro ao ler o arquivo:', err.message);
}
```

- **Eventos de Erro:**

```
const stream = fs.createReadStream('arquivo.txt');
stream.on('error', (err) => {
  console.error('Erro no stream:', err.message);
});
```

Depuração

1. Use `console.log()` para verificar variáveis.
2. Ferramentas como o `--inspect` para debugging interativo:

```
node --inspect app.js
```

2.6: Trabalhando com o módulo `http` para criar servidores básicos

Conceito

O módulo `http` permite criar servidores que recebem e respondem a requisições HTTP.

Exemplo Básico

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Olá, mundo!');
});

server.listen(3000, () => {
  console.log('Servidor rodando em http://localhost:3000');
});
```

Aplicabilidade

- Criação de servidores personalizados
- Entendimento do funcionamento interno do HTTP
- Base para frameworks como **Express.js**

 **Observação:** Frameworks abstraem essa complexidade, mas compreender o módulo `http` é essencial para entender como o backend funciona.

Lista de Exercícios

Questões Teóricas

- O que é o loop de eventos e qual sua importância no Node.js?
- Diferencie CommonJS de ES Modules.
- O que são Streams e Buffers?
- Liste três tipos de Streams e suas aplicações.

- Explique a diferença entre erros de runtime e erros de sintaxe.
- Quais são as vantagens de usar Streams para processamento de dados?
- Como o módulo `fs` pode ser usado para manipular arquivos?
- O que acontece quando um erro não é tratado em um stream?
- Qual é a função do `http` no Node.js?
- Explique o papel do Thread Pool no Node.js.

Questões Práticas

- Crie um servidor HTTP que responda com "Bem-vindo ao Node.js!".
- Escreva um código para criar e ler um arquivo usando o módulo `fs`.
- Implemente um stream para processar dados de um arquivo grande.
- Utilize Buffers para manipular uma string e convertê-la em dados binários.
- Crie um módulo CommonJS que exporta uma função simples e importe-o em outro arquivo.
- Configure e leia um arquivo JSON usando o módulo `fs`.
- Crie um script que utilize try/catch para tratar erros ao acessar um arquivo inexistente.
- Implemente um stream Duplex para modificar dados durante a leitura e escrita.
- Configure um servidor HTTP que responda de forma diferente para cada rota.
- Debug um erro em um script usando o `--inspect`.